Hybrid Feature-Based Detection of AI-Generated Text

Finn Fonteijn
s2899078
f.f.fonteijn@student.utwente.nl

Linn Gregussen
s3422372
l.y.gregussen@student.utwente.nl

Abstract

The proliferation of large language models necessitates reliable detection of AI-generated text in academic and journalistic contexts. This study investigates which linguistic features best distinguish human from AI-generated text (RQ1) and whether ensemble methods improve robustness (RQ2). We extract 40 linguistic and statistical features from 112,848 samples across four diverse datasets and train multiple classifiers. XGBoost achieves 97.62% accuracy (0.997 AUROC) on within-dataset testing, but performance drops to 82.76% on cross-dataset evaluation, revealing severe overfitting. In contrast, DeBERTa maintains 96.57% accuracy across datasets (0.994 AUROC), demonstrating superior generalization. Perplexity emerges as the most discriminative feature across all models (importance: 0.334-0.430), with higher perplexity indicating human text. Our voting ensemble achieves 95.58% within-dataset accuracy and 88.89% cross-dataset accuracy, providing robust performance through classifier diversity. Results demonstrate that transformer-based approaches generalize substantially better than feature-based methods for practical deployment, though simple ensemble voting further enhances robustness across domains.

1 Introduction

The availability of large language models (LLMs) creates a demand for reliable detection of AI-generated text in domains such as academia and journalism. Recent surveys indicate that hybrid approaches to AI detection, combining statistical and neural methods, often outperform single-modality detectors in both robustness and generalization (Wu et al., 2025; Su and Wu, 2024). A recent study by Erol et al. (2025) exemplifies these challenges in academic contexts, evaluating three commercial AI detectors on 250 human-written neurosurgery abstracts from pre-ChatGPT journals and 750 AI-generated versions (using ChatGPT 3.5, 4, and 4o). While the detectors achieved high AUC scores (0.96–1.00), they misclassified up to 30.4% of human texts as AI-generated, raising ethical concerns about false accusations in peer review.

Motivated by such findings, this project takes a dual approach to AI text detection. First, we conduct a feature-based analysis to identify which linguistic and statistical patterns are most discriminative. Second, we implement an ensemble-based detector combining multiple classification approaches to explore whether such combinations can improve practical deployment robustness.

This project addresses two research questions:

RQ1: *Which linguistic or statistical features are most effective in distinguishing human- vs. AI-generated text?*

RQ2: *How does ensemble-based classification improve robustness compared to individual classifiers?*

2 Related Work

Wu et al. (2025) conducted a comprehensive review of the state-of-the-art in LLM-generated text detection in 2025. They classify LLM-generated text detectors into three categories: watermarking technology where the generator includes identifiable patterns in the text to regulate and detect its origin; statistics-based detectors which analyze inherent patterns of the generated text; and neural-based detectors which use machine learning models to classify the generated text. The detectors in the survey report performance using different metrics, but among the best performing are the two statistics-based detectors "Ghostbuster" with average F1 score of 99.0 and GECScore with an AUROC of 98.62%. Among the neural-based detectors, DNA-GPT achieved "nearly 100%" detection accuracy in distinguishing between human-written and GPT-generated texts on multiple datasets. The watermarking methods are noted for their robustness against adversarial attacks, e.g. reliable classification when up to 50% of the tokens were modified, and low false-positive rates, making them a promising approach for reliable detection.

However, robustness against adversarial attacks remains a significant challenge. Weber-Wulff et al. (2023) examine the reliability of AI text detection

tools in academic settings and found that obfuscation techniques, e.g. manual editing and machine paraphrasing, significantly reduce the accuracy of detection tools, with approximately 50% of the texts that had undergone some type of post-generation editing being misclassified as human-written. Similar results, with detectors being less accurate when subjected to adversarial attacks, are reported by Wu et al. (2025) and Pudasaini et al. (2025).

Multiple surveys indicate that hybrid approaches, combining statistical and neural methods, often outperform single-modality detectors in robustness and generalization (Wu et al., 2025; Su and Wu, 2024).

3 Data

To ensure our model generalizes across diverse English language sources, domains, and LLMs, and to mitigate overfitting to any single dataset's biases, we utilize multiple datasets for training and evaluation. This approach allows for robust testing of features and hybrids in varied contexts. The datasets include AH&AITD (Arslan's Human and AI Text Database) with texts across domains including articles, abstracts, stories, news, and reviews (Akram, 2023); the LLM - Detect AI Generated Text Dataset (Kaggle sunilthite) containing essays (Thite, 2023); the DAIGT V2 Train Dataset (Kaggle) with essays (Deotte, 2023); and the Human ChatGPT Comparison Corpus (HC3) comprising Q&A from domains such as finance, medicine, law, and open-domain questions (Guo et al., 2023).

3.1 Dataset Balancing

The datasets showed varying degrees of class imbalance between the target classes human-written (0) and AI-generated (1), ranging from 31.5% AI-generated in the HC3 dataset to 50.0% AI-generated in the AH&AITD dataset. Dataset class distributions are detailed in Appendix A.

The dataset was balanced using a per-source strategy, where each source contributes an equal number of human-written and AI-generated samples according to its configured proportion of the total dataset size, and limited by the smallest class per source. This ensures balanced representation across both the source datasets and the target classes. Building on this balanced dataset foundation, we now describe our methodological approach for AI text detection, which combines feature-based analysis with ensemble-based classification.

4 Method

The project methodology consists of two efforts to address different aspects of AI text detection: a feature-based analysis and an ensemble-based detector.

4.1 Approach 1: Feature-Based Analysis

The feature-based analysis focuses on understanding which linguistic and statistical features are most informative in classifying human- vs. AI-generated text. For each text in the dataset, a set of features are extracted (section 4.1.1). These features are then used to train three lightweight classifiers: logistic regression and random forest using scikit-learn (Pedregosa et al., 2011), and XGBoost (Chen and Guestrin, 2016).

The dataset was balanced as described in section 3.1. Feature importance rankings were retrieved and compared across these three models, giving insights on which linguistic patterns are consistently important for distinguishing human- vs. AI-generated text.

4.1.1 Feature Extraction

For each entry in the dataset, the following features were extracted using the spaCy library (Honnibal et al., 2020) with the `en_core_web_sm` model for all linguistic processing tasks, including tokenization, lemmatization, part-of-speech tagging, and dependency parsing.

Lexical Features Four lexical features were chosen and extracted. Average word length and word length variance to measure vocabulary complexity. Root Type-Token Ratio (RTTR), and hapax legomena ratio to measure lexical diversity.

The Root Type-Token Ratio (RTTR) is a token-type ratio that accounts for document size by normalizing by the square root of the document length (Reviriego et al., 2024), defined as RTTR = Types / $\sqrt{\text{Tokens}}$. It was calculated by lemmatizing the text, counting the unique types and dividing by the square root of text length. A hapax legomenon refers to a word (or expression) occurring only once in a context, and here measures the proportion of words that appear exactly once in a sample.

Syntactic Features Sentence-level statistics were chosen to measure sentence complexity, and include average sentence length and variance.

Parse tree depth features (average, maximum and variance) were extracted to measure syntactic complexity. Parse depth is computed as the maximum number of ancestors for any token in a sentence.

Punctuation Features The ratio of punctuation marks to word count was calculated for comma, period, exclamation mark, and question mark. Additionally, the overall punctuation ratio was included.

Part-of-Speech Features POS tags were extracted using spaCy's coarse-grained POS tags (`token.pos_`) rather than fine-grained tags (`token.tag_`) to capture overall patterns in word class usage, fine grained tags were excluded as they may be too specific for the objective, and yield sparse data. The ratio of each POS category (noun, verb, adjective, adverb, etc.) to total tokens was calculated as well as a noun-to-verb ratio as a derived feature.

Perplexity Perplexity is defined as $\text{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(w_i)\right)$, where N is the number of tokens and $p(w_i)$ is the model's predicted probability for token i (Jurafsky and Martin, 2025). It measures how "surprised" the model is by the text: lower perplexity indicates more predictable text for the model. Perplexity was calculated using EleutherAI's Pythia 1.4B model (Biderman et al., 2023), an open-source model trained on the Pile dataset (Gao et al., 2020). Pythia was specifically chosen for its reproducibility and transparency in training, providing deduplicated data variants that ensure clean perplexity signals without contamination artifacts from memorized training data (Biderman et al., 2023). This makes it ideal for AI text detection where we need to distinguish true predictive patterns from artificial recognition of memorized content.

Burstiness Features In statistics, burstiness is a measure of the variability in data points over time. Burstiness captures the "clumpiness" in text structure, where AI text might show more uniform patterns while human writing exhibits greater variation. The core burstiness metric uses the Goh-Barabasi coefficient (Goh and Barabási, 2008): $B = \frac{\sigma-\mu}{\sigma+\mu}$, where σ is the standard deviation and μ is the mean, this measure is designed for analyzing variation in sequential data distributions, which is what we wish to analyze. The formula yields a burstiness score between -1 (uniform) and 1 (highly variable). For each text input, ten burstiness features were originally extracted, measuring variation at the sentence, word, and clause levels, including sentence length burstiness, positional variation (comparing first and second halves) and clause length burstiness. After correlation analysis, six burstiness features were retained.

Table 1: List of selected/considered linguistic features (40 features total). Dropped features (marked with †) were selected for redundancy while prioritizing informativeness.

Category	**Feature Name**
Lexical	average_word_length word_length_variance rttr_root_type_token_ratio hapax_legomena_ratio
Syntactic	average_sentence_length sentence_length_variance average_parse_depth maximum_parse_depth parse_depth_variance
Punctuation	comma_ratio period_ratio exclamation_mark_ratio question_mark_ratio punctuation_per_word
Part-of-Speech (POS)	pos_ratio_<tag> noun_to_verb_ratio
Burstiness	sentence_length_burstiness sentence_length_iqr position_burstiness_diff long_sentence_ratio word_length_burstiness clause_length_burstiness *sentence_length_cv*† *sentence_length_range_ratio*† *adjacent_sentence_variation*† *short_sentence_ratio*†
Other	perplexity

† Removed; high correlation ($r > 0.8$) with other feature(s)

4.1.2 Feature Selection

To reduce collinearity, a correlation analysis was performed on all the described extracted features, and from pairs with high pairwise correlation ($r > 0.8$), one feature was dropped. The feature to be dropped was selected based on whether it was correlated with more other features, and which feature would be more informative in the project context. After this selection process, 40 linguistic and statistical features were left for model training. The list of selected and dropped features is shown in Table 1, for the extended list with all POS-tags, see Appendix C.

4.2 Approach 2: Ensemble-Based Detector

The ensemble-based detector combines five classifiers to build a practical, robust AI text detector. The classifier outputs are combined using two methods: majority voting and stacking.

4.2.1 Base Classifiers

The ensemble-based detector is made up of multiple text-based classifiers and the XGBoost classifier from Approach 1 (Section 4.1) leveraging different perspectives on the classification task.

BERT Classifier BERT captures contextual word representations through its bidirectional transformer architecture, letting it capture semantic patterns in text. For this project the BERT classifier available at Hugging Face as `google-bert/bert-base-uncased` is used (Devlin et al., 2019).

DeBERTa Classifier A DeBERTa-based classifier using the `microsoft/deberta-v3-base` model. DeBERTa uses disentangled attention mechanisms and enhanced mask decoder, this way a word's content can be separated from its position and processed independently, this gives it a clearer sense of what's being said and where in the sentence it's happening, potentially capturing more nuanced linguistic patterns for the AI classification task (He et al., 2023).

Naive Bayes Classifiers Two Naive Bayes classifiers were included in the ensemble, one using the full vocabulary and one with stopwords removed.

XGBoost Classifier The XGBoost classifier described in section 4.1 was also added to the ensemble, giving the ensemble access to the explicit linguistic features extracted from the text.

4.2.2 Ensemble Methods

Two ensemble methods were implemented: majority voting and stacking.

Majority Voting In the voting approach, each of the five base classifiers independently predicts the class label for a given text. The final prediction of the ensemble is then determined by a majority vote across the classifiers. This simple aggregation reduces individual model errors, increasing robustness against the misclassification of a single model.

Stacking In the stacking approach, the predictions of all base classifiers on the training set are used as features to train a final estimator model. Three models were tested for this final estimator: Logistic Regression, Random Forest, and XGBoost, all implemented using the scikit-learn library (Pedregosa et al., 2011). Like majority voting, individual errors are reduced, but the use of a final estimator allows the ensemble to learn optimal weights for the different classifiers, potentially improving the overall performance of the ensemble.

4.3 Evaluation

4.3.1 Evaluation Strategy

To distinguish between optimistic within-distribution performance and real-world generalization, we employ two complementary evaluation approaches.

Within-dataset evaluation uses an 80/20 stratified train-test split on the combined balanced dataset, establishing performance upper bounds when training and deployment distributions align.

Cross-dataset evaluation employs leave-one-dataset-out cross-validation: training on three source datasets and testing on the fourth held-out dataset (10,000 samples per test). This stringent approach reveals whether models learn generalizable linguistic patterns or merely overfit to dataset-specific artifacts. The substantial distribution shift simulates real-world deployment where text sources are heterogeneous, providing a realistic assessment of practical reliability.

4.3.2 Classification Metrics

All classifiers are evaluated on a held-out test set using multiple performance metrics: accuracy, precision, recall, F1-score (macro, weighted, and per-class), and AUROC (Area Under the ROC Curve). AUROC provides a threshold-independent measure of classifier performance across all classification thresholds (0-1, with higher values indicating better discrimination between classes), with random baseline performance at 0.5. For the ensemble, both individual model performance and complete ensemble performance are reported, to allow comparison between the internal ensemble methods and the overall ensemble performance.

4.3.3 Feature Importance Analysis

For the feature-based analysis, we extract and compare feature importances from each model: absolute coefficient values from logistic regression (nor-

malized to sum to 1), Gini importance from random forest, and gain-based importance from XGBoost. By ranking features across models, we identify which linguistic patterns are consistently discriminative regardless of the classification algorithm.

SHAP Summary Plots SHAP (SHapley Additive exPlanations) values are calculated and visualized for the XGBoost classifier to provide interpretable explanations of feature contributions using the shap library (Lundberg, 2017). The SHAP summary plot visualizes the distribution and magnitude of each feature's impact across all test predictions by the XGBoost model, showing which features are most influential for the model's decisions, and in which direction.

5 Experimental Setup

The complete implementation of this project is available in the github repository nlp-ai-detector (Fonteijn and Gregussen, 2025).

5.1 Dataset Configuration

The combined dataset consisted of 112,848 samples from the datasets described in Section 3 and was balanced per source and per class as described in Section 3.1. Dataset composition is detailed in Appendix A. The data was split into training (80%) and test (20%) sets using stratified sampling with a fixed seed to ensure reproducibility and maintain class balance across splits, giving 90,278 training samples and 22,570 test samples.

5.2 LLM Setup

The LLM-models were downloaded from Hugging Face, and run using the transformers library (Wolf et al., 2020). The models were configured with the following hyperparameters:

Feature-Based Classifiers Logistic Regression used default scikit-learn parameters with random state 42. Random Forest was configured with max depth 10 and random state 42. XGBoost used 100 estimators, max depth 6, learning rate 0.1, random state 42, and eval metric 'logloss'.

Transformer-Based Classifiers BERT (`google-bert/bert-base-uncased`) and DeBERTa (`microsoft/deberta-v3-base`) classifiers were configured identically: maximum sequence length 128 tokens, 3 training epochs, learning rate 0.005, batch size 64. For both models, pre-trained embeddings were frozen and only a linear classification head was trained.

Metric	Logistic Regr.	Random Forest	XGBoost
Accuracy	0.928	0.961	**0.976**
AUROC	0.972	0.993	**0.997**
Human-Written			
Precision	0.931	0.949	**0.969**
Recall	0.925	0.973	**0.980**
F1-Score	0.928	0.961	**0.975**
AI-Generated			
Precision	0.926	0.973	**0.980**
Recall	0.931	0.948	**0.969**
F1-Score	0.928	0.960	**0.974**
F1 (Macro)	0.928	0.961	**0.975**
F1 (Weighted)	0.928	0.961	**0.975**

Table 2: Performance metrics for feature-based classifiers. Best values in bold.

Naive Bayes Classifiers Both Naive Bayes classifiers used alpha smoothing parameter 1.0 and bag-of-words features with CountVectorizer. The first used the full vocabulary, while the second had 'english' stopwords removed.

Ensemble Configuration The majority voting ensemble used unweighted voting across the five base classifiers. For stacking, all meta-classifiers Logistic Regression, Random Forest, and XGBoost, used library default parameters.

Perplexity Calculation Perplexity features were computed using the `EleutherAI/pythia-1.4b` model with a maximum sequence length of 1,536 tokens and a stride of 1,536 to avoid overlapping windows.

6 Results

6.1 Feature-Based Classifiers

Classification Performance Table 2 presents the performance of the three feature-based classifiers on the test set (22,570 samples, 11,285 per class). XGBoost achieved the highest performance with 97.62% accuracy and 0.997 AUROC, followed by Random Forest (96.06% accuracy, 0.993 AUROC) and Logistic Regression (92.82% accuracy, 0.972 AUROC).

Feature Importance Analysis Figures 1, 2, and 3 show the top 10 features for each classifier. Perplexity dominates across all models (importance 0.334-0.430), followed by lexical features (average word length, hapax legomena ratio). The models show differences in which type of features are more

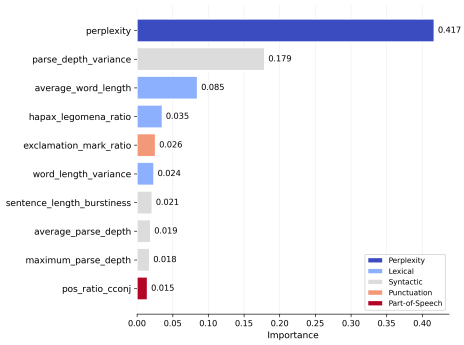

Figure 1: Top 10 feature importances for Logistic Regression classifier

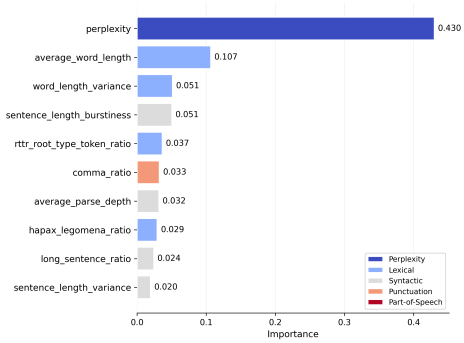

Figure 2: Top 10 feature importances for Random Forest classifier

discriminative: Logistic Regression and Random Forest emphasize syntactic complexity (four syntactic features each), while XGBoost relies more heavily on POS ratios (four POS features vs. one for LR and none for RF).

SHAP Summary Plot Figure 4 shows the SHAP analysis for the XGBoost classifier. Perplexity is the most discriminative feature, followed by lexical features (hapax legomena ratio, average word length, RTTR). High perplexity values (red) are strongly associated with human-written text, while low perplexity (blue) is strongly associated with AI-generated text. For the lexical features, lower values are associated with human-written text, that is less unique words, shorter words and a smaller range of words is associated with human-written text.

6.2 Ensemble Classifier

Table 3 presents the within-dataset performance of all classifiers in the ensemble-based detector on the held-out test set (22,570 samples). XGBoost achieved the highest individual performance (97.62% accuracy, 0.997 AUROC), followed by DeBERTa (94.24% accuracy, 0.988 AUROC) and

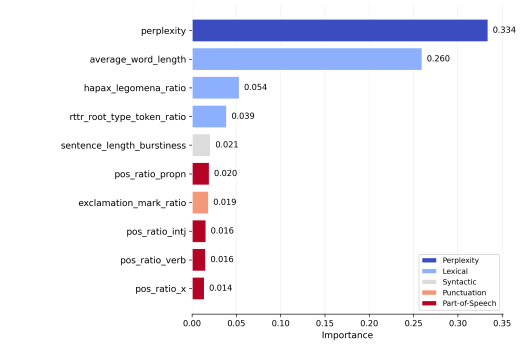

Figure 3: Top 10 feature importances for XGBoost classifier

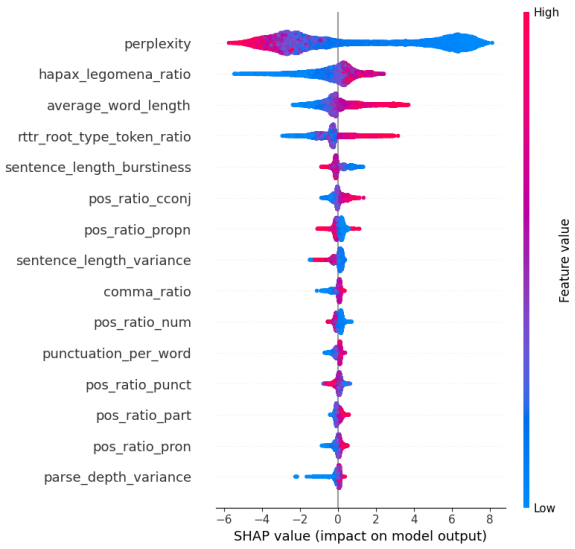

Figure 4: SHAP summary plot for XGBoost classifier. Positive values indicate AI-generated

BERT (91.33% accuracy, 0.972 AUROC). The voting ensemble achieved 95.58% accuracy, demonstrating robust aggregation across diverse classifiers, while the stacking ensemble matched XGBoost's performance (97.62% accuracy).

The near-perfect AUROC values (0.972-0.997) indicate excellent classification performance on the within-dataset test set, with XGBoost and DeBERTa demonstrating superior discrimination capability compared to baseline methods (see Appendix B for detailed AUROC curves).

6.3 Cross-Dataset Evaluation

As described in Section 4.3, we performed leave-one-dataset-out cross-validation, training on three datasets and testing on the held-out one (10,000 test samples per dataset). These results highlight a reversal in model rankings under distribution shifts. Table 4 compares within-dataset and cross-dataset

Classifier	Accuracy	Macro F1	AUROC
Random Classifier	0.501	0.501	0.500
Naive Bayes	0.860	0.860	0.925
BERT	0.913	0.913	0.972
DeBERTa	0.942	0.942	0.988
XGBoost	**0.976**	**0.976**	**0.997**
Voting Ensemble	0.956	0.956	N/A
Stacking Ensemble	**0.976**	**0.976**	N/A

Table 3: Within-dataset performance of ensemble component models. Best values in bold.

Model	Within Acc (%)	Cross Acc (%)	Change (%)	Variance
XGBoost	97.62	82.76	-14.86	±1.83
DeBERTa	94.24	96.57	+2.33	±0.18
BERT	91.33	89.26	-2.07	±2.52
Naive Bayes	86.00	80.95	-5.05	±2.14
Voting Ensemble	95.58	88.89	-6.69	±1.49
Stacking Ensemble	97.62	84.43	-13.19	±0.16

Table 4: Within-dataset vs. cross-dataset performance comparison.

performance, revealing significant changes in relative performance across models.

Table 5 provides detailed cross-dataset performance metrics for all classifiers. The transformer-based DeBERTa classifier achieved the highest performance (96.57% accuracy, 0.994 AUROC), demonstrating superior generalization across domains. Among the feature-based approaches, XGBoost showed significant overfitting with 82.76% accuracy compared to its within-dataset performance of 97.62% (Table 3), while the voting ensemble provided robust performance (88.89% accuracy).

AUROC curves on held-out datasets (Appendix B) confirm DeBERTa's superior discrimination (AUC 0.9948 and 0.9923) and XGBoost's limitations (AUC 0.7967 and 0.9224) when generalizing to unseen data.

The ensemble methods demonstrated robust performance across multiple evaluation metrics, with the voting ensemble achieving 88.89% accuracy (±1.49% variance) and providing consistent improvements in terms of stability and generalization across domains. Comparing to within-dataset results (Table 3), the voting ensemble maintained performance (+0.19% change), while the stacking ensemble declined (-4.27% change), suggesting that simple aggregation methods generalize better than complex meta-learning approaches.

Having examined both feature-based and ensemble performance, we now discuss the key findings and their implications for AI text detection.

Classifier	Accuracy	Macro F1	AUROC
Random Classifier	0.494	0.494	0.500
Naive Bayes	0.809	0.775	0.941
BERT	0.893	0.884	0.959
DeBERTa	**0.966**	**0.964**	**0.994**
XGBoost	0.828	0.814	0.860
Voting Ensemble	0.889	0.877	N/A
Stacking Ensemble	0.844	0.830	N/A

Table 5: Cross-dataset performance comparison of ensemble component models. Best values in bold.

7 Discussion and Conclusion

7.1 Feature Importance Findings

The consistency of perplexity as the top-ranked feature across all three models (importance scores: 0.334-0.430) indicates that perplexity captures fundamental differences in text generation patterns. As expected, the direction of this relationship is that a higher perplexity score is indicative of human-written text, i.e. that AI text is more predictable to the Pythia 1.4B model, likely because there are similarities in the training data of different LLMs.

Lexical features are also consistently among the top discriminators across all models. AI-generated text is characterized by longer words and greater vocabulary diversity. This shows that modern LLMs employ a more varied vocabulary than human writers, possibly reflecting their large and diverse training corpora, where the average human vocabulary reaches natural limits and domain constraints.

Apart from perplexity, the models show differences in the feature importance rankings. Logistic Regression and Random Forest emphasize syntactic complexity features (parse depth variance, sentence structure), while XGBoost relies on simpler POS-based features. However, cross-dataset evaluation (Table 4) reveals that XGBoost's high within-dataset performance (97.62% accuracy) stems from overfitting to dataset-specific artifacts, dropping to 82.76% on unseen data (as visualized in Figures 6 and 7, where its curve shows poorer separation with AUROC 0.7967–0.9224). This suggests that while linguistic features like perplexity remain discriminative, feature-based approaches may learn domain-specific patterns rather than general AI characteristics, limiting their real-world utility.

7.2 Ensemble-Based Detector vs. Individual Classifiers

Cross-dataset evaluation (Table 4) reveals a fundamental tradeoff: XGBoost excels within-dataset (97.62%) but overfits severely (-14.86 points to 82.76% cross-dataset), while DeBERTa generalizes better (94.24% to 96.57%), learning robust linguistic patterns rather than dataset artifacts (Figures 6 and 7). Voting ensembles provide middle-ground robustness (88.89%) with low variance (±1.49%), while stacking offers no benefit (84.43%), validating simple aggregation for RQ2. Feature-based methods capture dataset-specific patterns; transformers capture generalizable characteristics.

7.3 Method Limitations

Our datasets include only pure human-written or AI-generated samples, lacking mixed or adversarially modified text. Robustness to evasion techniques requires further evaluation before deployment. Additionally, datasets lack non-English text and text from latest LLMs (GPT-5, Claude Sonnet 4.5, Gemini 2.5 Pro), representing primarily academic essays and Q&A from pre-2024 models (GPT-3.5, GPT-4). Performance may degrade on contemporary text distributions (social media, code) and newer model generations, necessitating continuous retraining.

The perplexity feature uses a single model (Pythia 1.4B trained on the Pile), limiting generalizability to LLMs with different training distributions. Within-dataset evaluation overestimated XGBoost's effectiveness; cross-dataset testing proved essential for realistic assessment.

Computational Considerations: While transformers require more resources than feature-based approaches, XGBoost's computational advantage is limited by perplexity calculation requiring GPU-accelerated Pythia 1.4B inference. This accuracy-efficiency tradeoff matters for resource-constrained deployments.

7.4 Ethical and Societal Considerations

This project develops an AI detection tool, which can be used to detect AI-generated text in a range of domains such as academia, journalism or creative writing. Our intention is that knowledge about whether a text is AI-generated or not can be used in a "positive" way for society in general. In the mentioned domains, this could be help prevent fraud in academic settings, or help journalists flag sources that may be AI-generated and require further investigation. Our tool explicitly tackles the problem of adversarial attacks against AI-detection tools by using an ensemble of classifiers where one will have to fool multiple models to successfully fool the tool. However, it is important to reflect upon how such a tool can be used to improve and create adversarial attacks/tools, both against this specific tool, but also against AI-detectors in general, by training an adversarial model by using this tool in a detector-evasion setup.

Another ethical consideration is that this tool does not have zero false positive or false negative rates. Incorrect classification of human-written text can have serious consequences for the individual, such as a student being accused of cheating, possibly leading to failing a course or even being expelled from a university. Incorrect classification of AI-generated text on the other hand can lead to spread of misinformation such as "fake" news or made up scientific findings with a "stamp" of authenticity. To minimize these potential harms, a tool such as this should be deployed with transparent confidence scores and clear disclaimers and explanations about its limitations.

7.5 Conclusion

Perplexity and lexical features (average word length, hapax legomena ratio) are most discriminative for AI-text detection (RQ1), but transformers like DeBERTa (96.57% cross-dataset) generalize far better than feature-based XGBoost (82.76%), which overfits to dataset artifacts. Voting ensembles enhance robustness (88.89%) compared to individual classifiers (RQ2), though stacking offers no benefit. Cross-dataset evaluation proved essential—within-dataset metrics alone (XGBoost: 97.62%) misleadingly suggest reliability where severe overfitting exists.

For practical deployment, prioritize DeBERTa for cross-domain robustness; use XGBoost only when training and deployment distributions match. Cross-dataset validation is critical before deployment. Given ethical risks (false positives in academic settings, adversarial misuse), deploy with transparent confidence scores and human oversight. Future work should investigate adversarial robustness, domain adaptation, and why transformers generalize better than feature-based methods.

References

Arslan Akram. 2023. AH&AITD: Arslan's Human and AI Text Database. Dataset. Associated with the article: An Empirical Study of AI-Generated Text Detection Tools. Advances in Machine Learning & Artificial Intelligence, 4(2), 44–55.

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar Van Der Wal. 2023. Pythia: a suite for analyzing large language models across training and scaling. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org.

Tianqi Chen and Carlos Guestrin. 2016. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '16, page 785–794, New York, NY, USA. Association for Computing Machinery.

Chris Deotte. 2023. Daigt v2 train dataset. Dataset containing 44k human-written essays and AI-generated ones from different LLMs, including OSS ones.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *Preprint*, arXiv:1810.04805.

G. Erol, A. Ergen, B. Gülşen Erol, Ş. Kaya Ergen, T. S. Bora, A. D. Çölgeçen, B. Araz, C. Şahin, G. Bostancı, İ. Kılıç, Z. B. Macit, U. T. Sevgi, and A. Güngör. 2025. Can we trust academic ai detective? accuracy and limitations of ai-output detectors. *Acta neurochirurgica*, 167(1):214.

Finn Fonteijn and Linn Gregussen. 2025. Hybrid Feature-Based Detection of AI-Generated Text. GitHub repository.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2020. The Pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

K.-I. Goh and A.-L. Barabási. 2008. Burstiness and memory in complex systems. *Europhysics Letters*, 81(4):48002.

Biyang Guo, Xin Zhang, Ziyuan Wang, Minqi Jiang, Jinran Nie, Yuxuan Ding, Jianwei Yue, and Yupeng Wu. 2023. How close is chatgpt to human experts? comparison corpus, evaluation, and detection. *arXiv preprint arxiv:2301.07597*.

Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. Debertav3: Improving deberta using electra-style pretraining with gradient-disentangled embedding sharing. *Preprint*, arXiv:2111.09543.

Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. spaCy: Industrial-strength Natural Language Processing in Python. Zenodo.

Daniel Jurafsky and James H. Martin. 2025. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd edition. Draft. Online manuscript released August 24, 2025.

Scott M. Lundberg. 2017. Shap (shapley additive explanations). `https://github.com/shap/shap`. Accessed: 2025-06.

F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.

Shushanta Pudasaini, Luis Miralles-Pechuán, David Lillis, and Marisa Llorens Salvador. 2025. Survey on AI-Generated Plagiarism Detection: The Impact of Large Language Models on Academic Integrity. *Journal of Academic Ethics*, 23(3):1137–1170.

Pedro Reviriego, Javier Conde, Elena Merino-Gómez, Gonzalo Martínez, and José Alberto Hernández. 2024. Playing with words: Comparing the vocabulary and lexical diversity of chatgpt and humans. *Machine Learning with Applications*, 18:100602.

Yongye Su and Yuqing Wu. 2024. Robust detection of llm-generated text: A comparative analysis. *Preprint*, arXiv:2411.06248.

Sunil Thite. 2023. Llm - detect ai generated text dataset. Dataset containing more than 28,000 essays written by students and AI-generated.

Debora Weber-Wulff, Alla Anohina-Naumeca, Sonja Bjelobaba, Tomáš Foltýnek, Jean Guerrero-Dib, Olumide Popoola, Petr Šigut, and Lorna Waddington. 2023. Testing of detection tools for AI-generated text. *International Journal for Educational Integrity*, 19(1):26.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Junchao Wu, Shu Yang, Runzhe Zhan, Yulin Yuan, Lidia Sam Chao, and Derek Fai Wong. 2025. A survey on llm-generated text detection: Necessity, methods, and future directions. *Computational Linguistics*, 51(1):275–338.

Appendix

A Dataset Details

A.1 Dataset Class Distributions

Table 6 shows the original class distributions across the four datasets used in this study, before balancing.

Dataset	Samples	Human (%)	AI (%)
HC3	85K	68.5	31.5
sunilthite	29K	60.1	39.9
DAIGT V2	45K	61.0	39.0
AH&AITD	12K	50.0	50.0

Table 6: Dataset size and class distributions before balancing

A.2 Combined Dataset Composition

Table 7 shows the final composition of the combined balanced dataset used for training and evaluation.

Dataset	Samples	Proportion (%)
HC3	43,000	38.1
DAIGT V2	34,994	31.0
sunilthite	23,274	20.6
AH&AITD	11,580	10.3
Total	**112,848**	**100.0**

Table 7: Combined dataset composition after balancing

B AUROC Curves

B.1 Within-Dataset AUROC Curves

Figure 5 shows the AUROC curves for all classifiers on the within-dataset test set (22,570 samples). The curves illustrate the superior discrimination capability of XGBoost (AUC 0.997) and DeBERTa (AUC 0.988) compared to baseline methods. The near-perfect AUROC values indicate excellent classification performance when models are tested on data from the same distribution as training.

B.2 Cross-Dataset AUROC Curves

Figures 6 and 7 show AUROC curves on held-out datasets from leave-one-dataset-out cross-validation. These curves reveal significant performance differences when models face distribution shifts: DeBERTa maintains superior discrimination (AUC 0.9948 and 0.9923), while XGBoost shows substantial degradation (AUC 0.7967 and 0.9224), confirming its overfitting to training dataset characteristics.

C Full List of Extracted Features

`average_word_length`, `word_length_variance`, `rttr_root_type_token_ratio`, `hapax_legomena_ratio`, `average_sentence_length`, `sentence_length_variance`, `average_parse_depth`, `maximum_parse_depth`, `parse_depth_variance`, `comma_ratio`, `period_ratio`, `exclamation_mark_ratio`, `question_mark_ratio`, `punctuation_per_word`, `pos_ratio_adj`, `pos_ratio_adp`, `pos_ratio_adv`, `pos_ratio_aux`, `pos_ratio_conj`, `pos_ratio_cconj`, `pos_ratio_det`, `pos_ratio_intj`, `pos_ratio_noun`, `pos_ratio_num`, `pos_ratio_part`, `pos_ratio_pron`, `pos_ratio_propn`, `pos_ratio_punct`, `pos_ratio_sconj`, `pos_ratio_sym`, `pos_ratio_verb`, `pos_ratio_x`, `noun_to_verb_ratio`, `sentence_length_burstiness`, `sentence_length_iqr`, `position_burstiness_diff`, `long_sentence_ratio`, `word_length_burstiness`, `clause_length_burstiness`, `perplexity`

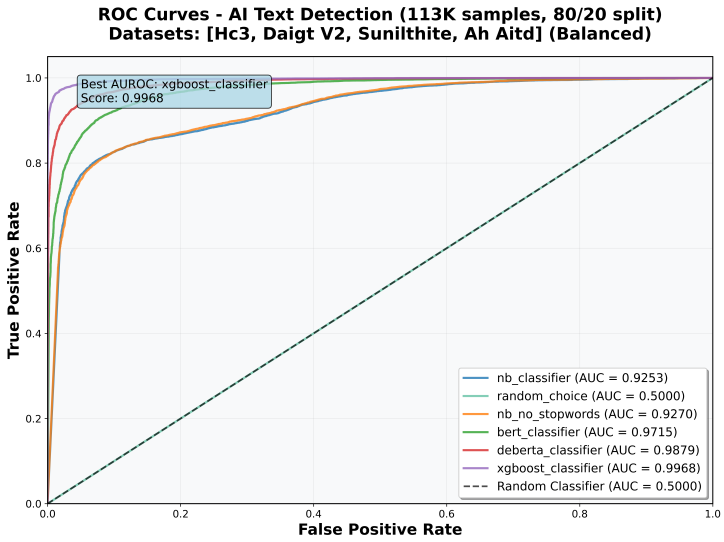

Figure 5: AUROC curves for all classifiers on within-dataset test set.

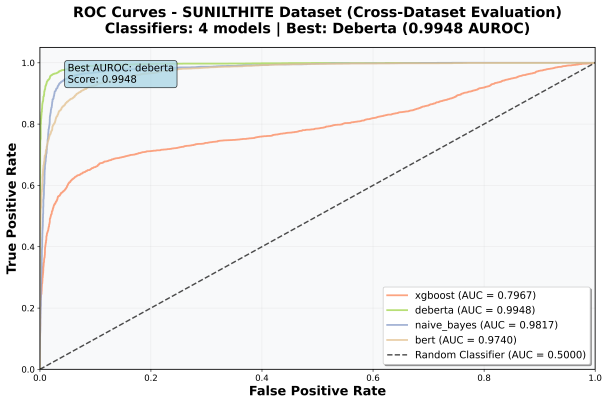

Figure 6: AUROC curves for models on the SUNILTHITE held-out dataset.

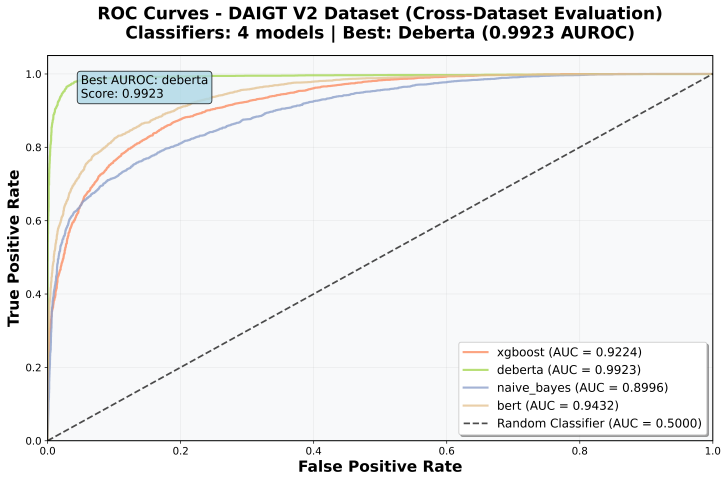

Figure 7: AUROC curves for models on the DAIGT V2 held-out dataset.